

JDBC

What is JDBC?

JDBC = Java Database Connectivity

It is a Java API used to connect a Java application with a database.

Java Program



JDBC



JDBC Driver



Database

Using JDBC, Java can communicate with databases such as:

- MySQL
- Oracle
- PostgreSQL
- SQL Server

Types of JDBC Drivers

Type	Name	Description
Type 1	JDBC-ODBC Bridge	Converts JDBC calls into ODBC calls
Type 2	Native API Driver	Uses database-specific native code
Type 3	Network Protocol Driver	Sends JDBC calls to middleware
Type 4	Thin Driver	Directly communicates with database

Type 4 – Thin Driver

This is the commonly used approach today.

Java Application



JDBC



MySQL JDBC Driver



MySQL

JDBC Packages

The two important JDBC packages are:

java.sql

Contains the main JDBC classes and interfaces

javax.sql

Provides additional database-related functionality

Important JDBC API Classes

DriverManager

Used to obtain a connection with the database

Connection con = DriverManager.getConnection(url, username, password);

Connection

Represents the connection between Java and the database

Connection con;

Statement

Used to execute SQL statements

Statement st = con.createStatement();

PreparedStatement

Used to execute parameterized SQL queries

PreparedStatement ps = con.prepareStatement("INSERT INTO student VALUES (?, ?)");

ResultSet

Stores the result returned by a SELECT query.

ResultSet rs = st.executeQuery("SELECT * FROM student");

JDBC Connection Steps

1. Import JDBC package



2. Load/Register driver



3. Establish connection



4. Create Statement



5. Execute SQL



6. Process ResultSet



7. Close connection

Basic Database Connection Program

```
import java.sql.*;
```

```
public class DBConnection {  
    public static void main(String[] args) {
```

```
        try {
```

```
String url = "jdbc:mysql://localhost:3306/college";
String user = "root";
String password = "1234";

Connection con =
    DriverManager.getConnection(url, user, password);

System.out.println("Database Connected Successfully");

con.close();

} catch (Exception e) {
    System.out.println(e);
}
}
```

import java.sql.*;

This imports the classes from Java's **JDBC API**.

java.sql contains classes/interfaces used for database operations.

Class / Interface	Purpose
DriverManager	Establishes a connection
Connection	Represents connection with database
Statement	Executes SQL queries
PreparedStatement	Executes parameterized SQL
ResultSet	Stores results returned by SELECT

Database Operations

These are commonly called **CRUD** operations:

CRUD	SQL	JDBC method
C — Create	INSERT	executeUpdate()
R — Read	SELECT	executeQuery()
U — Update	UPDATE	executeUpdate()
D — Delete	DELETE	executeUpdate()

student table creation

```
CREATE TABLE student(
    id INT,
    name VARCHAR(30)
);
```

Jar file inclusion procedure

Create a folder: Lib

JDBC jar file link

<https://dev.mysql.com/downloads/file/?id=513754>

Create a folder: .vscode

Create a file: settings.json

```
{
  "java.project.referencedLibraries": [
    "lib/**/*.jar"
  ]
}
```

SELECT Operation

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        try {
            // Connect to database
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "Test123"
            );

            System.out.println("Connected successfully");

            // Create statement
            Statement stmt = con.createStatement();

            // Execute query
            ResultSet rs = stmt.executeQuery(
                "SELECT * FROM student"
            );

            // Display records
            while (rs.next()) {
                System.out.println(
                    rs.getInt("id") + " " +
                    rs.getString("name")
                );
            }

            // Close connection
            con.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

INSERT Operation

```
import java.sql.*;

public class JDBCExample {
```

```
public static void main(String[] args) {

    try {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college",
            "root",
            "Test123"
        );

        Statement stmt = con.createStatement();

        // Insert a new record
        String sql = "INSERT INTO student VALUES (103, 'Bhumica')";

        int rows = stmt.executeUpdate(sql);

        System.out.println(rows + " record inserted successfully");
        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
```

Multiple INSERT Operation

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {

        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "Test123"
            );

            Statement stmt = con.createStatement();

            stmt.executeUpdate(
                "INSERT INTO student VALUES (104, 'Ravi')"
            );

            stmt.executeUpdate(
                "INSERT INTO student VALUES (105, 'Priya')"
            );

            System.out.println("Records inserted successfully");

            con.close();
        }
    }
}
```

```
} catch (Exception e) {  
    System.out.println(e);  
}  
}
```

UPDATE Operation

```
import java.sql.*;  
  
public class UpdateRecord {  
    public static void main(String[] args) {  
  
        try {  
            // Connect to database  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/college",  
                "root",  
                "Test123"  
            );  
  
            // Create Statement  
            Statement stmt = con.createStatement();  
  
            // Update record  
            String sql = "UPDATE student SET name = 'Ram' WHERE id = 100";  
  
            int rows = stmt.executeUpdate(sql);  
  
            // Display result  
            if (rows > 0) {  
                System.out.println("Record updated successfully");  
            } else {  
                System.out.println("Record not found");  
            }  
  
            // Close connection  
            con.close();  
  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

DELETE Operation

```
import java.sql.*;  
  
public class DeleteRecord {  
    public static void main(String[] args) {
```

```

try {
    // Connect to database
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/college",
        "root",
        "Test123"
    );

    // Create Statement
    Statement stmt = con.createStatement();

    // Delete record
    String sql = "DELETE FROM student WHERE id = 103";

    int rows = stmt.executeUpdate(sql);

    // Display result
    if (rows > 0) {
        System.out.println("Record deleted successfully");
    } else {
        System.out.println("Record not found");
    }

    // Close connection
    con.close();

} catch (Exception e) {
    System.out.println(e);
}
}

```

Complete Practical: Database Connections and Operations

```

import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {

        try {
            // 1. Connect to database
            Connection con =
                DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/college",
                    "root",
                    "1234"
                );

            System.out.println("Connected");

```

```
// 2. Create statement
Statement st = con.createStatement();

// 3. INSERT
st.executeUpdate(
    "INSERT INTO student VALUES (1, 'Arun')"
);

System.out.println("Record inserted");

// 4. SELECT
ResultSet rs =
    st.executeQuery("SELECT * FROM student");

System.out.println("Student Details:");

while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name")
    );
}

// 5. UPDATE
st.executeUpdate(
    "UPDATE student SET name='Ravi' WHERE id=1"
);

System.out.println("Record updated");

// 6. DELETE
st.executeUpdate(
    "DELETE FROM student WHERE id=1"
);

System.out.println("Record deleted");

// 7. Close connection
con.close();

} catch (Exception e) {
    System.out.println(e);
}
}
```

Using Prepared Statement

```
import java.sql.*;
```

```
public class JDBCExample {  
  
    public static void main(String[] args) {  
  
        try {  
  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/college",  
                "root",  
                "Test123"  
            );  
  
            String sql = "INSERT INTO student VALUES (?, ?)";  
  
            PreparedStatement ps = con.prepareStatement(sql);  
  
            ps.setInt(1, 103);  
            ps.setString(2, "Bhumica");  
  
            int rows = ps.executeUpdate();  
  
            System.out.println(rows + " record inserted successfully");  
  
            con.close();  
  
        } catch (Exception e) {  
  
            System.out.println(e);  
  
        }  
    }  
}
```

Statement vs PreparedStatement

```
Statement st = con.createStatement();  
  
st.executeUpdate(  
    "INSERT INTO student VALUES (1, 'Arun')"  
);  
  
PreparedStatement ps =  
    con.prepareStatement(  
        "INSERT INTO student VALUES (?, ?)"  
    );  
  
ps.setInt(1, 1);        //setInt(parameter_number, value)  
ps.setString(2, "Arun");  
  
ps.executeUpdate();
```

Method	Purpose
getConnection()	Connect to database
createStatement()	Create SQL statement
executeUpdate()	INSERT, UPDATE, DELETE
executeQuery()	SELECT
rs.next()	Move to next result row
rs.getInt()	Get integer value
rs.getString()	Get String value
close()	Close connection

Simple Login Page

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;

public class Student extends JFrame implements ActionListener {

    JLabel l1, l2;
    JTextField t1, t2;
    JButton b1;

    Student() {

        setTitle("Student Registration");
        setSize(400, 250);
        setLayout(new FlowLayout());

        l1 = new JLabel("Student ID:");
        l2 = new JLabel("Student Name:");

        t1 = new JTextField(20);
        t2 = new JTextField(20);

        b1 = new JButton("Save");

        add(l1);
        add(t1);

        add(l2);
        add(t2);

        add(b1);

        b1.addActionListener(this);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setVisible(true);
    }
}
```

```
public void actionPerformed(ActionEvent e) {

    int id = Integer.parseInt(t1.getText());
    String name = t2.getText();

    String url = "jdbc:mysql://localhost:3306/college";
    String username = "root";
    String password = "Test123";

    try {

        Connection con =
            DriverManager.getConnection(
                url, username, password
            );

        String sql =
            "INSERT INTO student VALUES (?, ?)";

        PreparedStatement ps =
            con.prepareStatement(sql);

        ps.setInt(1, id);
        ps.setString(2, name);

        ps.executeUpdate();

        JOptionPane.showMessageDialog(
            this,
            "Student inserted successfully"
        );

        t1.setText("");
        t2.setText("");

        con.close();

    } catch (Exception ex) {

        JOptionPane.showMessageDialog(
            this,
            ex.getMessage()
        );
    }

}

public static void main(String[] args) {

    new Student();
}
```



}

{